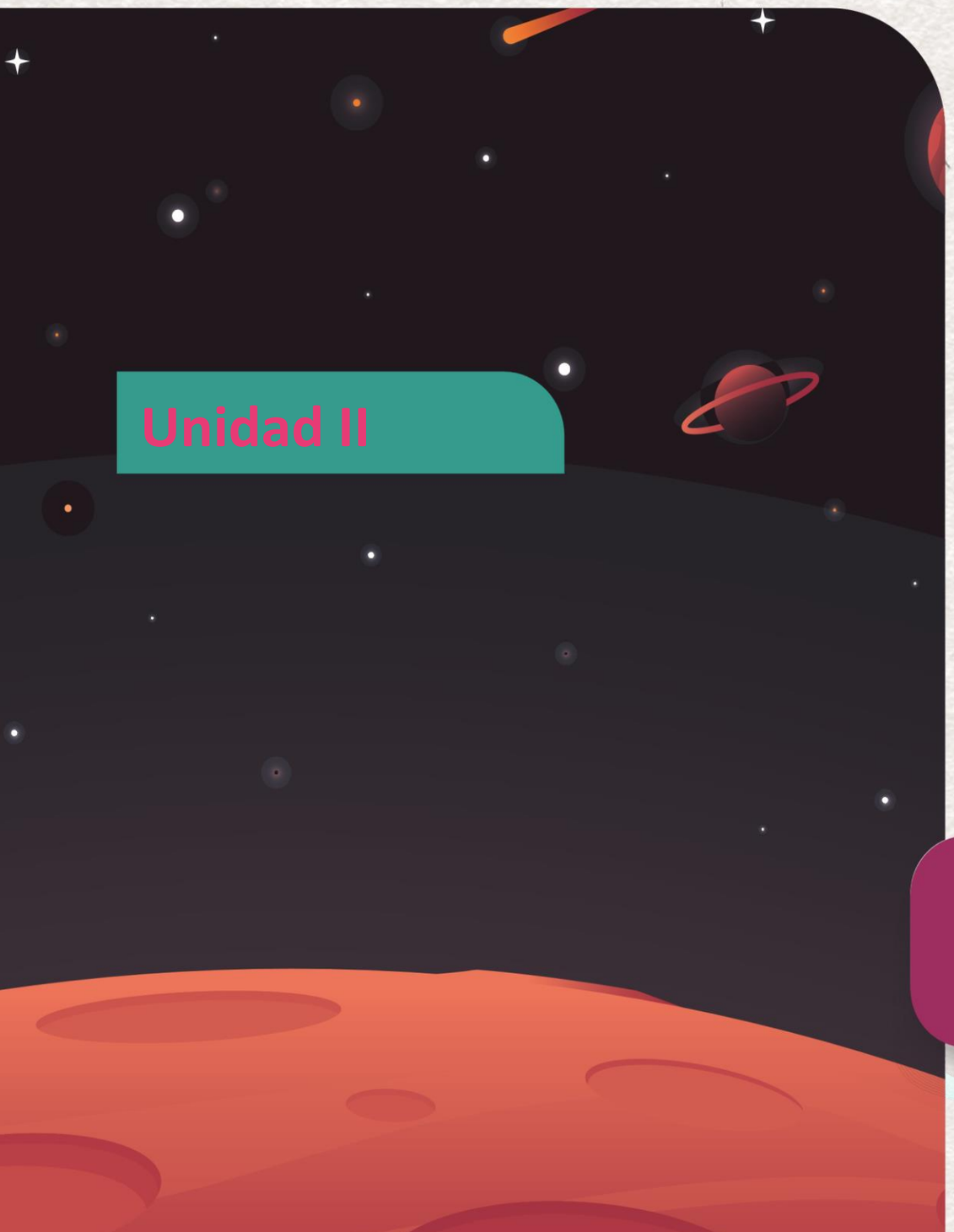
CESDE

Santiago Yosa

Ingeniero Sistemas de Información

CESDE | comfama



INTERFACES

- ¿Qué es una interfaz?

- Declaración de una interfaz

- Implementación de una interfaz uso de IMPLEMENTS

SISTEMA CONTROL DE VERSIONES

-Utilizar git/GitHub para versionar código realizado

Fundamentos de POO

Puente Cognitivo

Debes dominar el concepto de **Método Abstracto** (un método sin cuerpo que funciona como un contrato) y la imposibilidad de Java para realizar **Herencia Múltiple** con clases (una clase no puede tener dos clases padre usando extends).

Reto rápido

Lógicamente, un Smartphone es un DispositivoElectronico, pero también es una CamaraFotografica. Si intento compilar esto, ¿qué sucede y por qué?

```
public abstract class DispositivoElectronico {
   public abstract void encender();}
public abstract class CamaraFotografica {
   public abstract void tomarFoto();}

// El desarrollador intenta modelar la realidad:
public class Smartphone extends DispositivoElectronico, CamaraFotografica {
   // ...
}
```

Situación

En la clase anterior, modelamos jerarquías de objetos estrechamente relacionados (una Figura base para un Circulo y un Rectángulo). La clase abstracta funcionó de maravilla porque un Circulo **ES UN TIPO DE** Figura.

Ahora, imagina un sistema más grande. Tienes clases completamente diferentes:

- Una clase Usuario que representa a una persona en el sistema.
- Una clase Producto que representa un artículo en el inventario.
- Una clase Factura que representa una transacción de venta.

Estas tres clases no tienen ninguna relación de herencia entre sí. Un Usuario no es un Producto, y una Factura no es un Usuario. No tiene sentido crear una clase padre común para ellas.
Sin embargo, todas comparten una necesidad: **deben poder ser guardadas en la base de datos**.

Quieres una función GestorDeBaseDeDatos.guardar(objeto) que pueda tomar cualquiera de estos objetos y guardarlo. **¿Cómo garantizas que todos estos objetos, tan diferentes entre sí, tengan un método, digamos convertirAJson(), para que el gestor sepa cómo procesarlos?**
Una clase abstracta no puede resolver este problema, porque una clase solo puede heredar de **un** padre. Aquí es donde una **interfaz** brilla: es un contrato que cualquier clase, de cualquier jerarquía, puede prometer cumplir.

Clases Abstractas vs. Interfaces

Una **clase abstracta** es para una relación **"ES UN/A" (IS-A)**. Define la identidad y el código base compartido de una **familia** de clases (Circulo ES UNA Figura).

Una **interfaz** es para una relación **"PUEDE HACER" (CAN-DO)** o **"SE COMPORTA COMO"**. Define una **capacidad** o un **contrato** que clases no relacionadas pueden compartir (Usuario, Producto y Factura PUEDEN SER Guardable).

¿Qué es una Interfaz?

Es una referencia de tipo 100% abstracta. Es un **contrato** que define un conjunto de métodos abstractos y constantes.

Una interfaz no dice **cómo** hacer algo, solo **qué** se debe poder hacer.
Se declara usando la palabra clave interface.

Declaración e Implementación (interface e implements)

- Una clase no extiende una interfaz, la **implementa** usando la palabra clave implements.
- La regla de oro: una clase solo puede extends una superclase, pero puede implements **múltiples interfaces**. Esta es la forma en que Java logra la "herencia múltiple de tipo".

```
// Un Avion ES UN Vehiculo, y PUEDE HACER Volar y PUEDE SER Rastreado.
public class Avion extends Vehiculo implements Volador, Rastreable {
   // ...
}
```

Miembros de una Interfaz

Métodos: Por defecto, todos los métodos son public abstract. No necesitas escribir estas palabras clave.
Variables: Por defecto, todas las variables son public static final, es decir, **constantes**. Una interfaz no puede tener estado de instancia.

Constante

```
// Todas las variables en una interfaz son implícitamente public, static y final.
int VOLTAJE_OPERACION = 110; // Voltaje estándar en Colombia
```

Métodos Abstractos

```
// Definen el contrato principal. No tienen cuerpo. // Son implícitamente public y abstract.
void encender();
void apagar();
```

Método Default (desde Java 8)

```
// Proporciona una implementación por defecto que las clases pueden usar o sobrescribir.
default void modoAhorroEnergia() { System.out.println("Activando modo de bajo consumo..."); }
```

Método Static (desde Java 8)

```
// Es un método de utilidad que pertenece a la interfaz misma. // No puede ser sobrescrito y se llama directamente desde la interfaz.
static void mostrarAdvertenciaSeguridad() {
System.out.println("¡Peligro! Desconecte el dispositivo antes de abrirlo.");
}
```

La Evolución de las Interfaces (Java 8+)

Para mantener la relevancia, es crucial entender que las interfaces han evolucionado.

- **Métodos default:** Permiten añadir un método **con implementación** a una interfaz. ¿Por qué? Para poder agregar funcionalidad a interfaces existentes sin "romper" todas las clases que ya la implementaban. Es un mecanismo de evolución de APIs. La clase que implementa puede usar esta versión por defecto o sobrescribirla.
- **Métodos static:** Permiten añadir métodos de utilidad relacionados con la interfaz, que se pueden llamar directamente desde la interfaz (MiInterfaz.metodoUtil()) sin necesidad de una instancia.

Analogía: El Puerto USB

- El estándar **USB** es la **interfaz**. Define un contrato físico y de datos: "Cualquier dispositivo que quiera conectarse a un puerto USB debe tener este conector y hablar este protocolo".
- Un **ratón**, un **teclado** y una **memoria USB** son las **clases concretas**. Son objetos completamente diferentes, pero todos **implementan** la interfaz USB.
- El **puerto USB de tu computador** es el **código polimórfico**. No le importa qué dispositivo conectes; mientras cumpla con el contrato USB, sabrá cómo interactuar con él.

Preguntas

Si las interfaces ahora pueden tener métodos default con código, ¿cuál es la diferencia con una clase abstracta?

La **intención del diseño** sigue siendo la clave. Una **clase abstracta** define la naturaleza fundamental de una jerarquía de objetos y puede tener **estado** (variables de instancia) y **constructores**. Una **interfaz** define una **capacidad** que se puede "acoplar" a cualquier clase. Los métodos default son una herramienta para hacer esas capacidades más potentes o fáciles de evolucionar, pero no para definir la identidad de un objeto.

¿Por qué todas las variables en una interfaz son public static final?

Porque una interfaz es un contrato sin estado propio. No se pueden crear instancias de ella, por lo que las variables de instancia no tienen sentido. Las variables que se definen son constantes que pertenecen a la interfaz en sí misma (son static) y son parte del contrato público (public) que no se puede cambiar (final). Por ejemplo, la interfaz Math podría tener una constante public static final double PI = 3.14159;.

Ejemplo

```
// Archivo: Volador.java (El Contrato)
public interface Volador {
   // Métodos son public abstract por defecto
   void despegar();
   void volar();

   // Método default (desde Java 8)
   default void aterrizar() {
      System.out.println("Aterrizando de forma controlada.");
   }
}

// Archivo: Avion.java (Clase 1, no relacionada)
public class Avion implements Volador {
   @Override
   public void despegar() { System.out.println("El avión despega usando la pista."); }

   @Override
   public void volar() { System.out.println("El avión vuela con motores a reacción."); }
}

// Archivo: Pajaro.java (Clase 2, no relacionada)
public class Pajaro implements Volador {
   @Override
   public void despegar() { System.out.println("El pájaro salta y bate sus alas para despegar."); }

   @Override
   public void volar() { System.out.println("El pájaro planea en las corrientes de aire."); }

   // Opcional: puede sobrescribir el método default
   @Override
   public void aterrizar() { System.out.println("El pájaro aterriza suavemente en una rama."); }
}

// Archivo: Main.java
public class Main {
   public static void main(String[] args) {
      Volador miAvion = new Avion();
      Volador miPajaro = new Pajaro();

      miAvion.despegar();
      miAvion.volar();
      miAvion.aterrizar(); // Usa la implementación default

      System.out.println("---");

      miPajaro.despegar();
      miPajaro.volar();
      miPajaro.aterrizar(); // Usa su propia implementación
   }
}
```

Práctica Acompañada - Learning by Doing Seguridad de Datos

1. Crea una interfaz Encriptable con los métodos encriptar() y desencriptar(). Ambos deben retornar un String.
2. Crea la clase DocumentoSecreto que implemente la interfaz.
3. El atributo será textoPlano. Al llamar a encriptar(), retorna el texto añadiéndole la palabra "[CIFRADO] " al inicio. Al desencriptar, devuelve el texto normal.

```
public interface Encriptable {
   String encriptar();
   String desencriptar();
}

public class DocumentoSecreto implements Encriptable {
   private String textoPlano;

   public DocumentoSecreto(String texto) {
      this.textoPlano = texto;
   }

   @Override
   public String encriptar() {
      return "[CIFRADO] " + textoPlano; // Lógica simulada muy básica
   }

   @Override
   public String desencriptar() {
      return textoPlano;
   }
}
```

Práctica B: Afianzamiento y Git (Sistema de Bases de Datos)

Vamos a simular un entorno de desarrollo corporativo.

1. En tu repositorio, crea una nueva rama: git checkout -b feature/database-interface.
2. Crea una interfaz RepositorioBD con los métodos guardar(String dato) y leer().
3. Haz un commit: git commit -m "feat: creacion de interfaz RepositorioBD".
4. Crea dos clases que la implementen: RepositorioMySQL (imprime "Guardando en SQL...") y RepositorioMongo (imprime "Guardando en NoSQL...").
5. Haz otro commit: git commit -m "feat: implementacion de bases de datos SQL y NoSQL".
6. Empuja tu rama (git push origin feature/database-interface) y crea un Pull Request en GitHub.

```
public interface RepositorioBD {
    void guardar(String dato);
    void leer();
}

public class RepositorioMySQL implements RepositorioBD {
    @Override
    public void guardar(String dato) { System.out.println("Insert INTO tabla VALUES ('" + dato + "')"); }

    @Override
    public void leer() { System.out.println("SELECT * FROM tabla"); }
}

public class RepositorioMongo implements RepositorioBD {
    @Override
    public void guardar(String dato) { System.out.println("db.collection.insertOne({dato: '" + dato + "'})"); }

    @Override
    public void leer() { System.out.println("db.collection.find()"); }
}
```

Troubleshooting (Solución de Errores Comunes)

Errores de ejecución o sintaxis (Lectura de Consola):

1. **Error:** Cannot reduce the visibility of the inherited method from X
   - *Por qué ocurre:* En la interfaz, todos los métodos son public. Si al sobrescribir el método en tu clase se te olvida poner la palabra public (dejándolo como *default*), Java detecta que estás intentando bajar la seguridad del método de público a privado/paquete, lo cual está prohibido.
   - *Solución:* Asegúrate de que todos los métodos @Override de una interfaz sean explícitamente public.

2. **Error:** The type X must implement the inherited abstract method Y
   - *Por qué ocurre:* La clase dijo implements Interfaz, pero olvidaste escribir uno de los métodos requeridos. El contrato se incumple.

Ejercicio Independiente Aplicado

A. Situación: El Motor de Videojuego (Interfaces de Combate) Estás desarrollando un RPG. Los personajes tienen diferentes habilidades, pero no todos pueden volar ni todos pueden usar magia. Las interfaces dictarán qué puede hacer cada clase.

B. Instrucciones (Ticket de Tarea):

1. Crea una clase base abstracta PersonajeRPG con atributo nombre y un método abstracto atacar().
2. Crea la interfaz Mago con el método lanzarHechizo().
3. Crea la interfaz Volador con el método despegar().
4. Crea la clase Dragon que herede de PersonajeRPG e **implemente** Volador y Mago (¡Sí, puedes implementar múltiples interfaces separadas por comas!).
5. Crea la clase Caballero que herede de PersonajeRPG y **no implemente** ninguna de esas interfaces (solo ataca con espada).
6. Todo debe estar debidamente versionado en una rama de Git llamada feature/sistema-combate y fusionada (merged) a la rama main en GitHub.

C. Criterios de Logro Estrictos:

- **Sintáctico:** Uso correcto de sintaxis múltiple: class Dragon extends PersonajeRPG implements Mago, Volador.
- **Polimorfismo:** Todos los métodos exigidos por el padre y las interfaces deben tener su respectiva anotación @Override.
- **Auditoría Git:** El repositorio remoto debe mostrar la existencia del Pull Request y al menos 2 commits semánticos (feat: ...).

¿Debería saber algo más sobre el tema visto?

Interfaces Marcadoras (Marker Interfaces): Son interfaces que **no tienen ningún método**.

Su único propósito es "marcar" una clase como poseedora de una cierta capacidad. El ejemplo clásico es java.io.Serializable. Una clase que implements Serializable no tiene que implementar ningún método, solo le está diciendo a la JVM que sus objetos pueden ser convertidos a una secuencia de bytes. Se verifica con instanceof.

Interfaces Funcionales y Lambdas

Una interfaz con **un solo método abstracto** se llama **Interfaz Funcional** (anotación @FunctionalInterface). Son especiales porque puedes crear una instancia de ellas usando una sintaxis muy corta y potente llamada **expresión lambda**. Esto es la base de la programación funcional en Java.

CESDE | comfama